



Cat Simulator? Meow!

If Goat Simulator wasn't enough, a new Kickstarter campaign aims to raise \$20K to build a cat simulator game. <http://dgit.in/MeOW>



Image text editor

Project Naptha, a new browser extension for Chrome lets users edit text directly from and on an image itself. <http://dgit.in/ImgNaptha>

	Native	HTML5	Hybrid
App Features			
Graphics	Native APIs	HTML, Canvas, SVG	HTML, Canvas, SVG
Performance	Fast	Slow	Slow
Native look and feel	Native	Emulated	Emulated
Distribution	Appstore	Web	Appstore
Device Access			
Camera	Yes	No	Yes
Notifications	Yes	No	Yes
Contacts, calendar	Yes	No	Yes
Offline storage	Secure file storage	Shared SQL	Secure file system, shared SQL
Geolocation	Yes	Yes	Yes
Gestures			
Swipe	Yes	Yes	Yes
Pinch, spread	Yes	No	Yes
Connectivity	Online and offline	Mostly online	Online and offline
Development skills	ObjectiveC, Java, .NET	HTML5, CSS, Javascript, .NET	HTML5, CSS, Javascript, .NET

phone, there are workarounds for some of these native features now like iScroll which enables pinch-to-zoom and other gestures. JQuery Mobile is another framework which enables more features on HTML5 pages which look and feel like a native app.

The third approach is a hybrid between the above two approaches. There now exist frameworks which have a native codebase which interacts with the phone while the front end is made out of HTML5. These apps give you access to native features of each platform while exercising the flexibility of HTML5. The HTML5 webpages can be saved onto your phone (local storage) or hosted on a remote server (remote storage) with a few local pages just in case the internet connectivity is lost while the app is in use. The table below compares these three approaches:

Before you start!

You should prep for programming by getting hold of all the software needed for each approach. We'll be listing these at the beginning of each section. But before you can compile these projects you will need your developer certificate or signing key from each store. Certain tools will allow you to compile without a certificate but not for all platforms, Apple's system is a little too stringent and most tools won't create the package for iOS till you upload or provide the developer certificate. Android apps can be packaged and sideloaded onto your device to test without the certificate on most tools. Now there is no need for getting

a Mac if you wish to obtain an Apple developer certificate as is stated in the documentation. Check this link out for a way to do the same without a Mac. <http://dgit.in/iOSKey> For iOS, Android and Windows phone you need to register for a developer account and pay a certain registration fee to obtain your developer certificates.

Also, each individual app store has a certain review process and do restrict certain parameters. All of these things need to be adhered to if you wish to have your app approved. These guidelines can be obtained at <http://dgit.in/iOSRevi> and <http://dgit.in/AndGuide>.

Test, test and test!

While a simulator can run your apps effortlessly it is always advisable to have a device to test out your apps. If you include more native features then a proper feedback is only obtained if using a physical device. But if you simply wish to test your HTML5 website then you can use the emulator in Chrome to view your pages as you

work on them. Since Chrome uses Blink as the engine you will get a more similar look as obtained in Android devices. Blink is a fork of Webkit which powers iOS' UIWebView so the webpage will be rendered the same as on iOS.

Hybrid Approach

For this method you'll have to pick a framework to go with. There are a few frameworks but the popular ones are listed below.

- Corona
- PhoneGap
- Appcelerator

We'll build a simple HTML5 app using PhoneGap as a wrapper to provide access to native features of each platform.

Step 1 - Pick the resolution

A primary thing you need to keep in mind while building using HTML5 is that the very same layout will need to be rendered in the exact same way across all screen resolutions that exist. Refer the image below to figure out the most popular resolutions.

Keep in mind that the CSS for your website has to incorporate all these resolutions otherwise your app will look different on different devices. There are way too many resolutions listed here so pick the ones that are most popular. The following link shows the different resolutions of active android devices along with statistics highlighting how popular they are. <http://dgit.in/AndDash>

Step 2 - Code your website

You can use any tool to create your website, be it dreamweaver or kompozer or any of the numerous alternatives. However, if there are many includes in the web page then the app load time will increase. We tried our hand at creating a design using

	Low density (120), ldpi	Medium density (160), mdpi	High density (240), hdpi	Extra high density (320), xhdpi
Small screen	QVGA (240x320)		480x640	
Normal screen	WQVGA400 (240x400) WQVGA432 (240x432)	HVGA (320x480)	WVGA800 (480x800) WVGA854 (480x854) 600x1024	640x960
Large screen	WVGA800** (480x800) WVGA854** (480x854)	WVGA800* (480x800) WVGA854* (480x854) 600x1024		
Extra Large screen	1024x600	WXGA (1280x800) [†] 1024x768 1280x768	1536x1152 1920x1152 1920x1200	2048x1536 2560x1536 2560x1600

Ensure that your app scales well according to the device resolution